Introduction

Nuclear power plants are vulnerable to cyber attacks since the Stuxnet attack revealed that computer malware could harm the hardware regardless of whether the computers are connected to the Internet. There are federal laws (10 CFR 73.54 and NRC Regulatory Guide 5.71) that mandate nuclear plants to secure their digital infrastructure.

The objectives of AEGIS-OS are:

1. Only allow legit code to run

2. Parts are segregated so one compromised part doesn't harm the entire system

3. Safety of the power plant in case of emergencies

The reactor's emergency shut-down system remains independent and hard-wired, thus no software failure will hinder the emergency shutdown system.

1. System Architecture and Information Flow

The architecture of AEGIS-OS consists of several layers, illustrated by Image 1. The lowest layer is the security hardware – a TPM chip, memory encryption, and IOMMU that regulates the access to devices. Next comes the seL4 kernel, which is the sole software component controlling the machine. All other elements run in four isolated partitions: Reactor Control, Plant Monitoring, Authentication (access badges and biometrics), and Regulator View (read-only access for government officials).

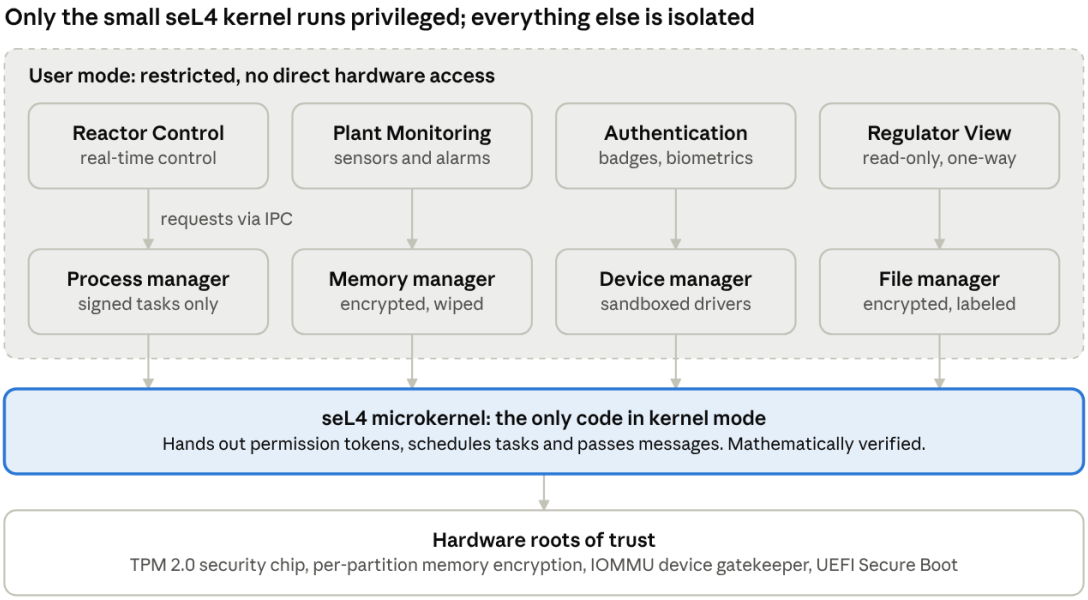


Image 1: Architecture of AEGIS-OS: four partitions, four OS services, one kernel

The four OS components are located between the partitions and the kernel:

1. Process Manager: launches/terminates processes; launches only authorized programs.
2. Memory Manager: assigns encrypted memory to each partition; cleans the memory after each use.
3. Device Manager: launches each device driver in the sandbox environment; a faulty driver will not be able to interfere with other components.
4. File Manager: stores files on the encrypted disk drive; performs a permission check whenever a file is accessed.

Memory is not shared among the partitions. Communication occurs through the controlled channels approved by the kernel.

From startup to operation: when power is turned on, the code in the chip verifies the firmware; the firmware verifies the bootloader; the bootloader verifies the kernel; and the kernel verifies each OS component before launch. The corresponding log is also created in the TPM chip, and the disk key is released only if all the logs match. Then the server of the regulator confirms the authenticity of the system, and the control of the reactor commences.

2. Secure System Design Principles

Three principles drive the design process. Least privilege principle states that every component must have the minimum necessary level of permissions. The fail-safe default principle means that no permissions will be given unless explicitly stated, and any failures will result in a secure state. Defense in depth requires several protection layers to exist.

1. Process management – programs have only the permissions stated in their signed files, all unknown programs cannot be executed, and in case a process crashes, it is replaced by a backup process.
2. Memory management – each partition can only access its own memory, data cannot be used to execute code, and several protection mechanisms exist to prevent memory attacks
3. Device management – each driver can only access its own device, all unknown devices are blocked, and communication between the device and computer is encrypted.
4. File management – each program can access only the files from its own partition, corrupted files are rejected, and backups are kept off-line.
5. Secure boot – only signed code can be booted, and any failed verification prevents booting; everything is recorded in the TPM.

3. Underlying Operating System

The underlying operating system of AEGIS-OS is a microkernel called seL4. The microkernel design has only the most essential functionality (memory management, task scheduling, and access control) implemented in the kernel itself, while everything else runs as individual processes with reduced privileges. seL4 has been formally verified, which means that it is free from many typical bugs. Moreover, it is tiny, which makes seL4 easy to audit and guarantees timely completion of time-critical tasks.

seL4 fulfills all requirements: it provides guaranteed execution time for real-time tasks, ensures strict memory protection, keeps drivers out of the kernel, and is compatible with regular Secure Boot and TPM chips. Other non-time-critical functions are performed by an enhanced Linux OS, which uses the reliable ext4 file system with disk encryption.

Some alternative candidates were QNX and INTEGRITY-178, which are utilized in aircrafts and other safety-critical systems. Both solutions are solid, but the code is proprietary and unverified.

4. Types of Data

1. Process logs: logs of people who initiated or terminated processes, instructions of operators, and alarms. They cannot be edited by anyone since they are immutable.
2. Memory states: current reactor state, encryption keys, and inter-process communication messages. They are encrypted and purged after their usage.
3. Device I/O data: information provided by sensors like temperature, pressure, radiation, and commands delivered to devices. They flow across encrypted and validated channels.
4. File meta-data: file owner, authorized people to access the file, and the file's hash value. Any changes in them require approval from two parties.
5. Boot and firmware integrity data: TPM logs, signatures and versioning. They are kept in the TPM and protected memory, and can be altered only through signed modifications.

5. Access Control Mechanisms

AEGIS-OS uses permission tokens for programs, system-enforced labels for files, and job roles for people (such as Operator, Supervisor, Engineer, or Regulator). Any action that affects safety needs two people to approve it.

Area	Access control method
Process creation and termination	The process manager alone may run programs; only signed programs are allowed. Killing a reactor process requires a supervisor and another person. Each partition has its own limit of programs that can be started.
Memory access rights	Memory pages have attributes of being readable, writable, or executable. No page is writable and executable at the same time.
Device permissions	Only Reactor Control can send commands to equipment. The regulator has no device access.
File access rights	Files are labeled by partition. Logs can only be added to, never changed, even by administrators.
Secure boot verification	Only code signed with the facility's key runs. Code on the blocked list is always refused.

6. Cryptographic Functions

All encryption uses government-approved standards. Plant equipment is used for decades, so AEGIS-OS uses strong keys and plans ahead for future quantum computers.

Use	Choice	Why
Memory encryption	AES, built into the processor, with a separate key for each partition	Fast enough for real-time work, and stops anyone from reading the memory chips directly
Device channel protection	TLS 1.3 (the same protection used by secure websites) with AES-256	Keeps sensor data private and detects any tampering
File system encryption	AES-256 full-disk encryption on ext4 (similar to BitLocker on NTFS and FileVault on APFS)	A stolen drive cannot be read, because the key is locked in the TPM
Bootloader signature validation	RSA or ECDSA signatures, plus a newer quantum-resistant signature (ML-DSA)	Works with Secure Boot today and stays secure in the future

7. Authentication Methods

Who or what	How it proves its identity
Users	Smart card, PIN, and fingerprint. Safety actions need two operators. Remote regulators use hardware security keys.
Processes	Each program is identified by its digital signature. A changed program no longer matches and is not trusted.
Devices	Each sensor or device has a digital certificate built into its chip. Unknown devices are blocked and an alarm goes off.
File-level permissions	Access depends on both the user's role and which program is asking.
Secure boot	The firmware checks signatures against the facility's own certificates. The TPM signs a report of the boot so the regulator can confirm it remotely.

8. Data Leakage Prevention and Isolation

1. Process isolation – each partition has its own memory and its own quota for processor usage. For regulating communication link there is a one-way device, which allows data to pass through in one direction.
2. Memory sandboxing – a partition has access only to its own memory, each partition has its own encryption key, and memory is erased before it is reused.
3. Encrypted device communication – all device communication is encrypted, each device has access to its own memory segment, and USB memory devices are disabled.

4. Access limitations for file system – files are marked with partitions and users cannot change markings. Two persons are needed to copy data.

9. Protection Against Attacks and Misuse

Threat	What it is	How AEGIS-OS stops it
Process hijacking	An attacker takes over a running program	Only signed programs run, and each has very limited permissions
Fork bomb	A program copies itself until the system runs out of resources	Each partition has a hard limit on programs and processor time
Buffer overflow	A program writes too much data and overwrites nearby memory	The memory protections in Section 12, plus safer programming languages
Race condition	Two tasks change the same data at once and corrupt it	Partitions do not share memory, and updates happen all at once or not at all
Rogue device	An unknown device is plugged in	Unapproved devices are blocked and logged, and ports are locked
DMA attack	A device reads memory directly, skipping the processor	The IOMMU only lets each device reach its own memory area
File corruption	Data is damaged by an attack, a bug, or a power loss	Files are checked on every read, and signed backups are kept
Misconfigured ACLs	A permission setting gives too much access by mistake	Permissions come from an approved policy and are audited weekly
Bootkit	Malware that loads before the OS	Secure Boot refuses any unsigned startup code
Rootkit	Malware hidden inside the kernel	A small, proven kernel, signed drivers only, and remote checks

10. System Integrity

It means that all code and configuration is officially approved. Secure boot establishes the chain of trust; it works similarly to the line of guards who check IDs of those who follow:

1. Firmware – the code in the chip verifies the firmware.
2. Bootloader – the firmware verifies the bootloader by means of Secure Boot.
3. Kernel – the bootloader verifies the seL4 kernel.
4. OS services – the kernel verifies each OS service before starting it.

In the process, fingerprints of each next stage are stored on the TPM chip. They can only be deleted after rebooting.

During the operation, the audit logs register all logins, commands, and alerts. Each log entry is connected with the previous one, which means that any change will be detected. The watchdog ensures the reaction of the critical processes and activates a backup in case of failure. The devices are verified both during connection and operation.

11. Secure Boot

Secure boot is the foundation of the whole design. If an attacker could change the startup process, they could turn off every other protection. AEGIS-OS uses two methods together: verified boot blocks untrusted code, and measured boot records what ran so it can be proven later (Image 2).

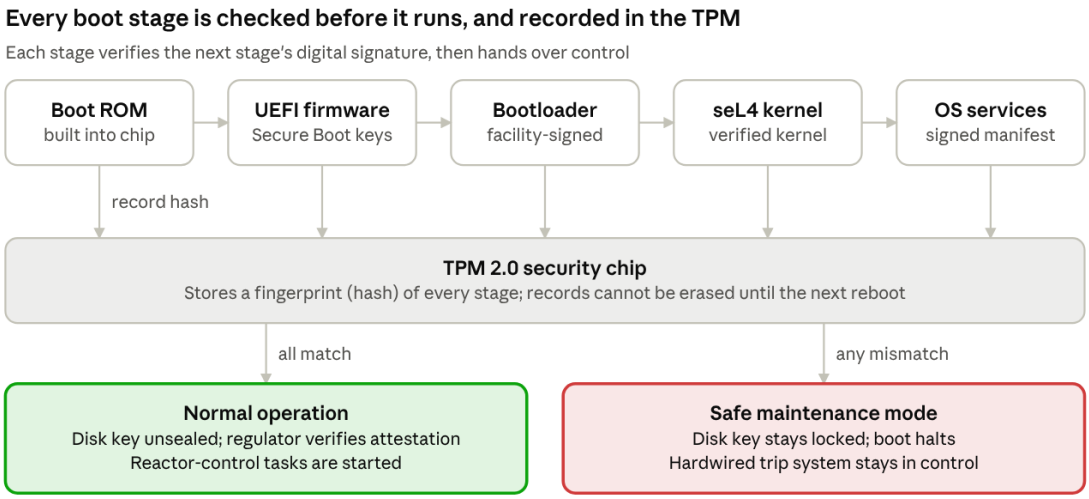


Image 2: Secure boot chain of trust: five verified stages, two outcomes

- Checking the bootloader: the firmware keeps a list of trusted keys and a list of blocked code. A bootloader runs only if it is signed with a trusted key and is not on the blocked list. AEGIS-OS uses the facility's own key, which is kept offline and used only with two people present.
- Kernel and driver checks: the bootloader checks the kernel's signature before loading it. Drivers are ordinary signed programs, so they are checked the same way. The system disk is also checked as it is read, so later changes are caught too.
- Firmware protection and rollback prevention: firmware is stored in protected memory, and only signed updates are allowed. To stop attackers from installing an older version with known bugs, the TPM keeps a version counter that can only go up.
- UEFI Secure Boot, TPM sealing, and the trusted computing base: UEFI Secure Boot is the firmware feature that checks signatures. TPM sealing means the disk key is only released if the boot records match. The trusted computing base is the small set of parts that must be trustworthy (the chip, TPM, firmware, bootloader, and kernel), so AEGIS-OS keeps it as small as possible.

12. Memory Management

Code is only loaded during startup when its signature is verified. Subsequently, all memory is allocated by the memory manager so that all active memory blocks are initialized by trusted code. Devices cannot access memory unless it is properly initialized.

Some of the common threats against memory include buffer overflow and stack smashing (write beyond memory boundary to gain control of the process), memory corruption (modify data like reactor settings), and memory reuse (attack on the memory that has been released already). AEGIS-OS protects against them using:

1. ASLR: Programs are allocated memory randomly on each launch so that attackers cannot predict their location.
2. KASLR: Kernel memory allocation is done similarly on each startup.
3. DEP: Memory is marked either as code or as data but not as both, which it so injected data cannot be run as code
4. Guard pages: Empty and inaccessible pages located around memory blocks that will kill any program that tries to access them.
5. Memory wiping and encryption: Memory is wiped clean and encrypted individually for each partition.

13. Process Management

Process refers to an active program. The entire lifecycle of the process is controlled by AEGIS-OS:

1. Creation – only signed programs are started, with only the privileges allowed to them.
2. Suspension - suspending a safety function requires two users, and a backup is initiated first.
3. Termination – upon terminating a program, its memory is erased and all its privileges revoked. In case of failure of a critical task, a pre-checked backup version replaces it.

List of programs which run at boot time is included in the signed system image. Any attempt to add malware to the list would make the signature invalid, hence the system will not boot.

There are two states of the processor. The kernel state runs all the hardware, while the user state can only use what is made available to it. In AEGIS-OS, only the seL4 microkernel is running in the kernel state, while every other program runs in the user state and requests everything from the kernel. As only the kernel can distribute the privileges, a compromised program cannot escalate its privileges.

Conclusion

AEGIS-OS is built around one idea - the secure boot should be the priority. This means that everything starts from a clean, secure place because only the code that is checked is allowed to boot up. With a small, well-known kernel, separation, encryption, and dual control, one error or a piece of malware will not take control over the whole plant.

References

1. Cybersecurity and Infrastructure Security Agency. (2018). *Primary Stuxnet advisory* (ICSA-10-272-01). https://www.cisa.gov/news-events/ics-advisories/icsa-10-272-01
2. International Atomic Energy Agency. (2021). *Computer security techniques for nuclear facilities* (IAEA Nuclear Security Series No. 17-T, Rev. 1). https://www.iaea.org/publications/14729/computer-security-techniques-for-nuclear-facilities
3. Microsoft. (2025, August 18). *Secure the Windows boot process*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/security/operating-system-security/system-security/secure-the-windows-10-boot-process
4. Microsoft. (2026, August 18). *Trusted Platform Module technology overview*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/security/hardware-security/tpm/trusted-platform-module-overview
5. OWASP Foundation. (n.d.). *Buffer overflow*. Retrieved November 9, 2026, from https://owasp.org/www-community/vulnerabilities/Buffer_Overflow
6. Protection of Digital Computer and Communication Systems and Networks, 10 C.F.R. § 73.54. https://www.ecfr.gov/current/title-10/chapter-I/part-73/subpart-F/section-73.54
7. Regenscheid, A. (2018). *Platform firmware resiliency guidelines* (NIST Special Publication 800-193). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-193
8. seL4 Foundation. (n.d.). *About seL4*. Retrieved November 9, 2026, from https://sel4.systems/About/
9. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley. https://www.wiley.com/en-us/Operating+System+Concepts%2C+10th+Edition-p-9781119320913
10. Stouffer, K., Pease, M., Tang, C., Zimmerman, T., Pillitteri, V., Lightman, S., Hahn, A., Saravia, S., Sherule, A., & Thompson, M. (2023). *Guide to operational technology (OT) security* (NIST Special Publication 800-82r3). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-82r3
11. Ubuntu. (n.d.). *UEFI/SecureBoot*. Ubuntu Wiki. Retrieved November 9, 2026, from https://wiki.ubuntu.com/UEFI/SecureBoot
12. U.S. Nuclear Regulatory Commission. (2023). *Cyber security programs for nuclear power reactors* (Regulatory Guide 5.71, Revision 1). https://www.nrc.gov/docs/ML2225/ML22258A204.pdf